SQL Injection - Basics

Automated Exploitation and Database Manipulation

SQL Injection (SQLi) is a critical web vulnerability that occurs when an application improperly handles user-supplied data, allowing an attacker to interfere with the queries made to its database. By injecting malicious **SQL** code, attackers can bypass authentication, access sensitive information, or even modify data. This module explores the mechanics of SQLi, focusing on the lack of **input sanitization** and validation. It introduces **SQLMap**, a powerful automated tool used to detect and exploit these flaws across various techniques like **Boolean-based**, **Error-based**, and **UNION-based** injections. Through systematic enumeration, security professionals can identify vulnerable parameters and secure data against unauthorized extraction.

This **documentation** and **learning** approach is strongly inspired by the structured, hands-on content provided by [TryHackMe](), which significantly contributed to practical understanding of "Sql Injection Basics" concepts.

DOCUMENTED BY

Kaustubh Ghosalkar aka "WEIRDO"

LINKEDIN GITHUB

1. Understanding SQL Injection (SQLi)

Websites interact with databases using **DBMS** (Database Management Systems) like MySQL or PostgreSQL via **Structured Query Language (SQL)**. A vulnerability arises when user input is concatenated directly into a query without proper **Sanitization**.

Authentication Bypass Logic

A standard login query looks like this:

SELECT * FROM users WHERE username = 'John' AND password = 'Password123';

An attacker can bypass the password check by injecting logic that makes the statement **Always True**.

- **Injected Input:** ' OR 1=1;-- -
- **Resulting Query:** SELECT * FROM users WHERE username = 'John' AND password = 'abc' OR 1=1;-- -';

The **OR 1=1** condition is always true, and the **-- -** (comment) ignores the rest of the original query, granting unauthorized access.

2. Automated Tool: SQLMap

SQLMap is an open-source tool that automates the process of detecting and exploiting SQL injection flaws. It is the industry standard for database takeover.

Detection Techniques

SQLMap tests for several types of SQLi:

- **Boolean-based blind:** Uses "True/False" questions to extract data bit by bit.
- **Error-based:** Forces the database to reveal information through descriptive error messages.
- **UNION query-based:** Uses the UNION operator to combine the results of an attacker's query with the original.
- **Time-based blind:** Uses time delays (e.g., SLEEP(5)) to determine if a query was successful.

WEIRDO

3. SQLMap Command Reference

SQLMap requires a target URL or an intercepted request to begin its analysis.

Core Flags and Operations

Flag	Name	Purpose
-u	**URL**	Defines the target URL (HTTP GET testing).
--wizard	**Wizard**	Interactive guided mode for beginners.
--dbs	**Databases**	**Enumerates** all available database names.
-D [DB] --tables	**Tables**	Lists tables within a specific database.
-T [Table] --dump	**Dump**	Extracts and displays all records from a table.
--cookie	**Cookie**	Passes session tokens for authenticated testing.
-r [File]	**Request**	Uses an intercepted text file (HTTP POST testing).

4. Advanced Enumeration and Scanning

In many scenarios, a basic scan might not find the vulnerability. Increasing the **Level** and **Risk** allows SQLMap to perform a deeper, more aggressive analysis.

- **--level=5:** Increases the number of payloads and test points (e.g., testing inside Headers).
- **--risk=3:** Increases the risk of the payloads (use with caution on production systems).

5. Practical Task (Task 4)

I researched the login page of the target application and **found** that it used GET parameters, although they were not initially visible in the address bar. Below is how **I solved** the exploitation process:

- **Work Done: I practiced** using the browser's **Network Tab** (Inspect -> Network) to capture the full request. By entering test credentials and clicking login, **I founded** the hidden parameters in the request URL: http://MACHINE_IP/ai/includes/user_login?email=test&password=test.
- **Solution: I practiced** running SQLMap against this URL. To ensure success, **I solved** the detection issue by adding the --level=5 flag for an in-depth scan.
- **Database Extraction: I practiced** the enumeration chain:
    1. **I found** the database names using --dbs.
    2. **I researched** the "ai" database and **found** the table names using -D ai --tables.
    3. **I solved** the final goal by dumping the user table using -D ai -T [table_name] --dump.
- **Result: I founded** the administrator's password and successfully retrieved the flags required for the challenge.

WEIRDO

Conclusion and Cheatsheet

Conclusion

SQL Injection remains one of the most dangerous web vulnerabilities because it provides direct access to the "crown jewels" of an organization—the data. While manual exploitation is possible through logical manipulation like **' OR 1=1**, automated tools like **SQLMap** make the identification and extraction process significantly more efficient. Security practitioners must prioritize **Parameterization** and **Input Validation** to prevent these attacks, ensuring that user input is never interpreted as executable code by the database.

Fast Revision Cheatsheet

- **SQLi:** Exploiting a lack of **Input Sanitization** to run arbitrary SQL code.
- **' OR 1=1;-- -**: The classic "Always True" authentication bypass.
- **SQLMap -u**: The primary command to start a GET-based scan.
- **--dbs**: List all databases.
- **--tables**: List all tables in a selected DB.
- **--dump**: Extract data from a table.
- **--level=5**: Maximum depth for testing parameters.
- **GET vs POST:** Use -u for URL parameters; use -r for body-based data.
- **WAF/IPS:** Use --check-waf in SQLMap to see if a firewall is blocking you.